**Evrlink System Overview:**

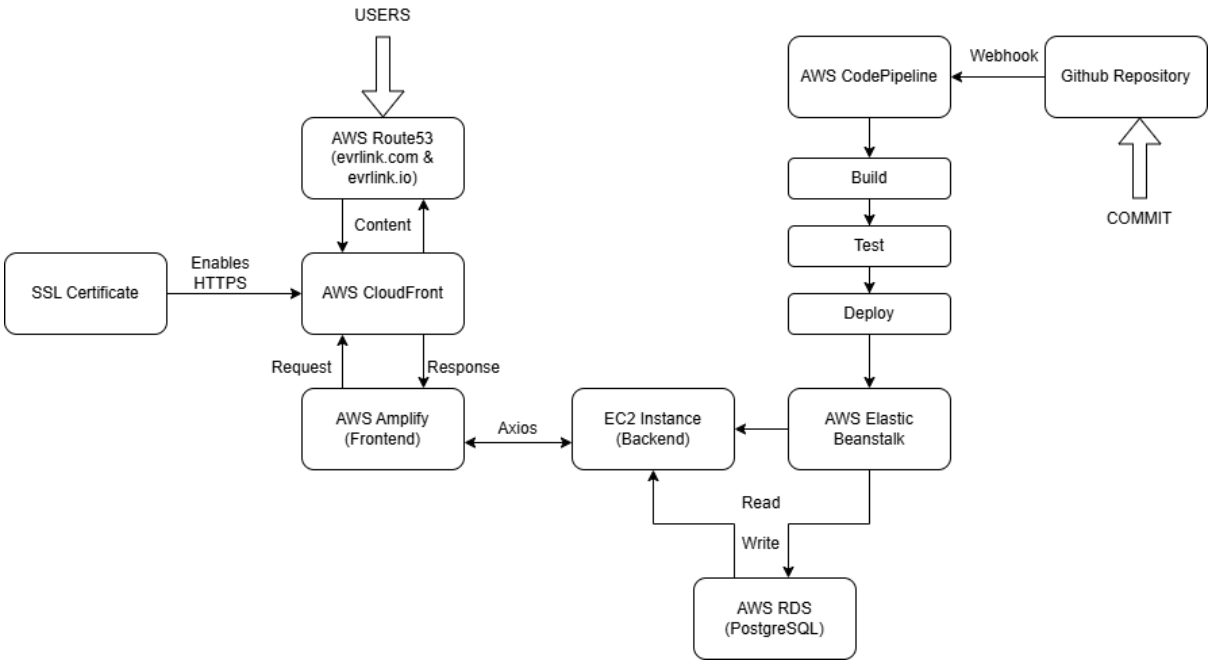
1. **User Onboarding & Wallet Management**
    - Register with email and associate wallet address (/api/auth/email-wallet, /api/user)
    - Receive cards via email(link) and prompt wallet creation
2. **Greeting Card Creation & Personalization**
    - Create personalized greeting cards, select backgrounds, add messages (/api/artnfts, /api/giftcard/create)
    - Mint cards as NFTs and send to wallet
    - Choose from categories/templates (/api/backgrounds, /api/backgrounds/category/:category)
3. **Card Sending & Receiving**
    - Send cards to wallet addresses, base usernames, or via email/QR (/api/giftcard/transfer, /api/giftcard/transfer-by-baseusername)
    - Receivers can view, claim, and see their collection (/api/profile/:walletAddress, /api/giftcards/owner/:address) Status: Implemente d (API supports sending, claiming, viewing collections)
4. **Payments & Monetization**
    - Pay for cards with USDC, ETH.
    - Artists receive a share of sales; platform retains a percentage (monetization logic implied)
5. **User Experience & Guidance**
    - Clear steps, helpful prompts, user-friendly terminology (user stories highlight need) Status: UX guidance is a frontend concern; backend supports flows
6. **Transaction & Activity Tracking**
    - Track card transactions and user activities (/api/giftcard/:id/transactions, /api/users/:walletAddress/activity)
7. **ENS/CB.ID Resolution**
    - Send cards to ENS/base usernames, resolved to wallet addresses (/api/resolve-cbid) Status: Implemented
8. **Image & NFT Management**
    - Upload images for cards and backgrounds (/api/image/upload) Status: Implemented
9. **Chatbot & Agent Support**
    - Chatbot and agent endpoints for user support or on-chain interactions (/api/chatbot/message, /api/agent) Status: Implemented
10. **No Secret Phrase for Receivers**

- Receivers don’t need a secret phrase to unlock cards (user story)

**Basic Flow:**

1. **User Authentication:** User logs in or creates a wallet.
2. **Action Initiation:** User initiates an action (e.g., send meep) via the UI or API.
3. **Transaction Creation:** The system constructs and signs the transaction using the Server Wallet **(TO BE CHANGED)**
4. **Transaction Submission:** Transaction is sent to the blockchain via the Blockchain Connector.
5. **Data Synchronization:** Data Sync Module writes blockchain-confirmed data into the database and updates the local state accordingly.
6. **Feedback:** UI/API provides real-time status and results to the user.

**Application System Architecture:**




**1. Code & Pipeline**

- **GitHub Repository**
  → Source code repository containing frontend and backend code.

- **AWS CodePipeline**
  → Automates the release process.
    - **Build**
    - **Test**
    - **Deploy**

---

**2. Frontend Hosting**

- **AWS Amplify**
  → Hosts the frontend application (React, Angular, etc.).
- **AWS CloudFront**
  → CDN for delivering frontend content globally with low latency.
- **SSL Certificate**
  → Provides HTTPS encryption for secure communication.
- **AWS Route 53 (evrlink.com & evrlink.io)**
  → DNS service managing the domain and routing traffic to CloudFront/Amplify.

---

**3. Backend Hosting**

- **EC2 Instance (Backend)**
  → Hosts the backend application (e.g., Node.js/Express server).
- **AWS Elastic Beanstalk**
  → Manages the deployment of backend app on EC2 with auto-scaling and monitoring.

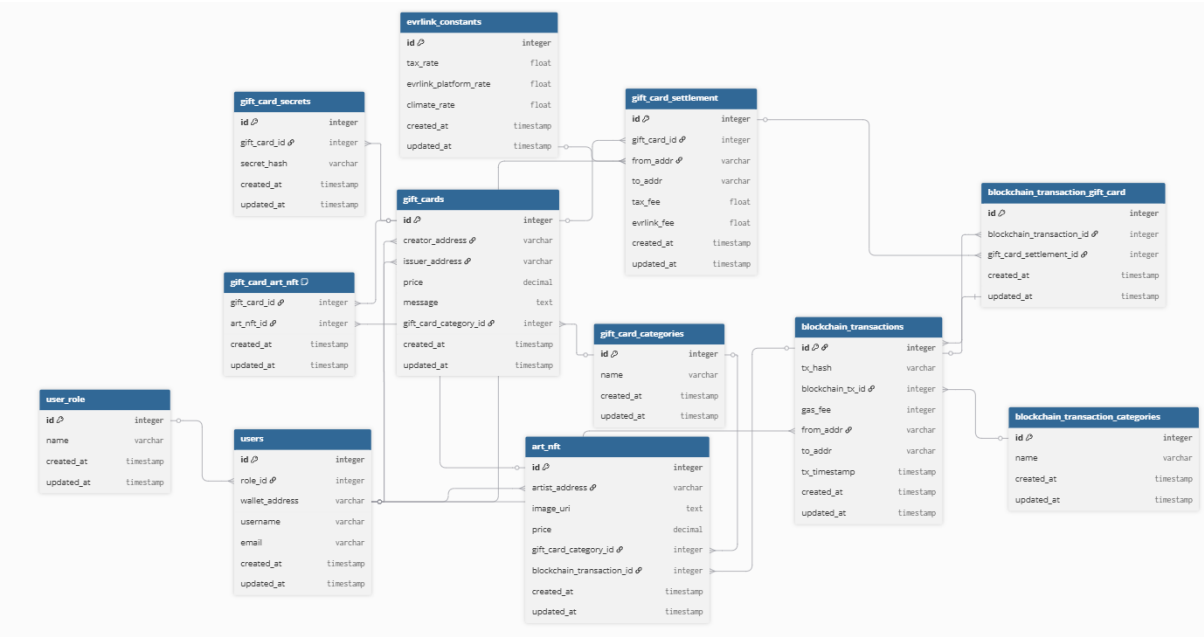
---

**4. Database Layer**

- **AWS RDS (PostgreSQL)**
  → Manages the relational database used by the backend.

---

**Data Model:**




***DBML:***

```
Table users {
  id            integer [pk]
  role_id       integer [ref: > user_role.id]
  wallet_address varchar
  username      varchar
  email         varchar
  created_at    timestamp
  updated_at    timestamp
}

Table user_role {
  id        integer [pk]
  name      varchar
  created_at timestamp
  updated_at timestamp
}

Table gift_cards {
  id                  integer [pk]
```

```
  creator_address       varchar [ref: > users.wallet_address]
  issuer_address        varchar [ref: > users.wallet_address]
  price                 decimal
  message               text
  gift_card_category_id integer [ref: > gift_card_categories.id]
  created_at            timestamp
  updated_at            timestamp
}

Table gift_card_categories {
  id         integer [pk]
  name       varchar
  created_at timestamp
  updated_at timestamp
}

Table gift_card_secrets {
  id           integer [pk]
  gift_card_id integer [ref: > gift_cards.id]
  secret_hash  varchar [unique]
  created_at   timestamp
  updated_at   timestamp
}

Table art_nft {
  id                    integer [pk]
  artist_address        varchar [ref: > users.wallet_address]
  image_uri             text [unique]
  price                 decimal
  gift_card_category_id integer [ref: > gift_card_categories.id]
  blockchain_transaction_id integer [ref: > blockchain_transactions.id]  // New FK
  created_at            timestamp
```

```
  updated_at            timestamp
}

Table gift_card_art_nft {
  gift_card_id integer [ref: > gift_cards.id]
  art_nft_id   integer [ref: > art_nft.id]
  created_at   timestamp
  updated_at   timestamp

  Note: "Associates gift cards to multiple art NFTs"

  indexes {
    (gift_card_id, art_nft_id) [unique]
  }
}

Table gift_card_settlement {
  id          integer [pk]
  gift_card_id integer [ref: > gift_cards.id, unique]
  from_addr    varchar [ref: > users.wallet_address]
  to_addr      varchar
  tax_fee      float
  evrlink_fee  float
  created_at   timestamp
  updated_at   timestamp
}

Table blockchain_transactions {
  id                integer [pk]
  tx_hash           varchar [unique]
  blockchain_tx_id     integer [ref: > blockchain_transaction_categories.id]
  gas_fee           integer
```

```
  from_addr           varchar [ref: > users.wallet_address]
  to_addr             varchar
  tx_timestamp        timestamp
  created_at          timestamp
  updated_at          timestamp
}

Table blockchain_transaction_gift_card {
  id                          integer [pk]
  blockchain_transaction_id   integer [ref: > blockchain_transactions.id]
  gift_card_settlement_id     integer [ref: > gift_card_settlement.id]
  created_at                  timestamp
  updated_at                  timestamp
}

Table blockchain_transaction_categories {
  id          integer [pk]
  name        varchar [unique]
  created_at timestamp
  updated_at timestamp
}

Table evrlink_constants {
  id                  integer [pk]
  tax_rate            float
  evrlink_platform_rate float
  climate_rate        float
  created_at          timestamp
  updated_at          timestamp
}
```

**API Documentation:**

---

**Evrlink Backend API Documentation**

This document describes the available REST API endpoints for the Evrlink NFT Gift Marketplace backend.

---

**Authentication**

**POST /api/auth/email-wallet**

Associates an email with a wallet address or updates user information.
**Request body:**

- walletAddress (string)
- email (string)
- user_name (string)
- role_id (number)

**Response:** 201 Created or 200 OK with user info.

**GET /api/auth/email-wallet?email=...**

Fetches wallet address by email.

---

**Email-Wallet Association (Legacy)**

**POST /email-wallet**

Associates an email with a wallet address (legacy using raw SQL).

**GET /email-wallet?email=...**

Fetches wallet address by email (legacy using raw SQL).

---

**Agent**

**POST /api/agent**

Sends a message to the on-chain agent.
**Request body:**

- message (string)
- userId (optional string)

**Response:** JSON with the agent's reply.

---

**Art NFTs / Backgrounds**

**POST /api/artnfts**

Mints a new Art NFT (background).
**Form data:**

- image (file, required)
- priceUsdc (number, required)
- artistAddress (string, required)
- giftCardId (string, required)
- category (string, optional)

**GET /api/backgrounds**

Returns a list of backgrounds. Supports pagination and category filter.

**GET /api/background/:id**

Returns background details by ID.

**GET /api/backgrounds/category/:category**

Returns backgrounds by a specific category.

**GET /api/backgrounds/popular**

Returns popular backgrounds.

**GET /api/backgrounds/test**

Test endpoint for backgrounds.

---

**Gift Cards**

**POST /api/giftcard/create or /api/gift-cards/create**

Creates a new gift card.
**Request body:**

- backgroundId, price, message, creatorAddress, artNftId, secret, recipientAddress

**POST /api/giftcard/price**

Calculates the ETH required to mint a gift card.
**Request body:**

- backgroundId, price

**GET /api/giftcards**

Lists all gift cards with pagination and filters.

**GET /api/giftcard/:id**

Fetches a gift card by its ID.

**GET /api/giftcards/owner/:address**

Fetches gift cards owned by a specific wallet address.

**GET /api/giftcards/creator/:address**

Fetches gift cards created by a specific wallet address.

**POST /api/giftcard/transfer**

Transfers a gift card to another wallet.
**Request body:**

- giftCardId, recipient

**POST /api/giftcard/transfer-by-baseusername**

Transfers a gift card using the recipient's base username.
**Request body:**

- giftCardId, baseUsername

**POST /api/giftcard/claim**

Claims a gift card using a secret.
**Request body:**

- giftCardId, secret, claimerAddress

**POST /api/gift-cards/:id/set-secret**

Sets or updates the secret for a gift card.
**Request body:**

- secret, ownerAddress, artNftId (optional)

---

**Transactions**

**GET /api/giftcard/:id/transactions**

Returns all transactions for a gift card.

**GET /api/transactions/recent**

Returns recent gift card transactions.

---

**Users**

**POST /api/user**

Registers or updates a user.
**Request body:**

- walletAddress, username, email, roleId, bio, profileImageUrl

**GET /api/user/:walletAddress**

Returns a user profile with detailed stats.

**DELETE /api/user/:walletAddress**

Deletes a user by wallet address.

**GET /api/users/top**

Returns top users based on activity.

**GET /api/users/search?query=...**

Searches users by username, email, or wallet.

**GET /api/users/:walletAddress/activity**

Returns the activity feed for a user.

**GET /api/users**

Returns all users with pagination and sorting.

---

**Profiles**

**GET /api/profile/:walletAddress**

Returns a user profile with received and sent gift cards.

---

**Images**

**POST /api/image/upload**

Uploads an image for a background or gift card.

---

**ENS/CB.ID Resolution**

**POST /api/resolve-cbid**

Resolves a base username (cb.id) to a wallet address.
**Request body:**

- name (string)

---

**Chatbot**

**POST /api/chatbot/message**

Sends a message to the chatbot.
**Request body:**

- message (string)

---

**NOTE:**

See README.md for project structure and local setup instructions.